

TASK 1 / ЗАВДАННЯ 1

BASIC DATA RETRIEVAL IN MYSQL: THE DEEP SPACE REGISTRY

БАЗОВЕ ВИБИРАННЯ ДАНИХ У MYSQL: РЕЄСТР ГЛИБОКОГО КОСМОСУ

SELECT, SELECT DISTINCT, LIMIT, ORDER BY.

Environment: MySQL 8.0+ (MySQL Workbench or the mysql command-line client).

SELECT, SELECT DISTINCT, LIMIT, ORDER BY.

Середовище: MySQL 8.0+ (MySQL Workbench або клієнт командного рядка mysql).

Scenario · Сценарій

The Orbital Administration maintains a registry of ships assigned to deep-space expeditions. Each record stores the ship name, its class, the sector of its home port, crew size, launch year and the remaining fuel reserve in per cent. Your task is to answer four queries from the flight-control officer using only data-retrieval statements.

Орбітальне управління веде реєстр суден, приписаних до експедицій у глибокий космос. Кожен запис містить назву судна, його клас, сектор порту приписки, чисельність екіпажу, рік запуску та залишок палива у відсотках. Завдання - відповісти на чотири запити диспетчера польотів, використовуючи лише оператори вибирання даних.

0. Environment setup · Підготовка середовища

The statements below are outside the current topic and are given ready to copy and paste. Execute the whole block once. CREATE DATABASE creates the storage container, USE selects it as the default schema for the session, CREATE TABLE defines the structure (DDL), and INSERT loads twelve records. The column ship_id is generated automatically by AUTO_INCREMENT.

Наведені нижче оператори виходять за межі поточної теми і подані у готовому до копіювання вигляді. Виконайте блок один раз. CREATE DATABASE створює контейнер зберігання, USE обирає його як стандартну схему сеансу, CREATE TABLE визначає структуру (DDL), а INSERT завантажує дванадцять записів. Стовпець ship_id формується автоматично завдяки AUTO_INCREMENT.

```
DROP DATABASE IF EXISTS deep_space;
CREATE DATABASE deep_space CHARACTER SET utf8mb4;
USE deep_space;

CREATE TABLE starship (
  ship_id      INT AUTO_INCREMENT PRIMARY KEY,
  ship_name    VARCHAR(40) NOT NULL,
  ship_class   VARCHAR(20) NOT NULL,
  home_sector  VARCHAR(20) NOT NULL,
  crew_size    SMALLINT UNSIGNED NOT NULL,
  launch_year  YEAR NOT NULL,
  fuel_reserve DECIMAL(5,2) NOT NULL
) ENGINE = InnoDB;

INSERT INTO starship (ship_name, ship_class, home_sector,
  crew_size, launch_year, fuel_reserve)
VALUES ('Aurora Borealis', 'Explorer', 'Orion Arm', 120, 2231, 87.50),
('Kraken', 'Freighter', 'Cygnus Rift', 45, 2228, 64.20),
('Pathfinder', 'Scout', 'Orion Arm', 12, 2240, 95.10),
('Iron Comet', 'Freighter', 'Perseus Gap', 50, 2225, 41.75),
('Nova Lance', 'Cruiser', 'Cygnus Rift', 210, 2238, 72.40),
('Silent Drift', 'Scout', 'Perseus Gap', 10, 2242, 88.00),
('Helios Dawn', 'Explorer', 'Orion Arm', 135, 2236, 55.30),
```

```
( 'Vega Runner',      'Freighter', 'Orion Arm',    48, 2233, 30.90),
( 'Titan Wake',       'Cruiser',   'Perseus Gap', 198, 2229, 67.85),
( 'Ember Sail',       'Explorer',  'Cygnus Rift', 110, 2244, 99.00),
( 'Ghost Signal',     'Scout',     'Cygnus Rift',  14, 2241, 76.60),
( 'Solar Anvil',      'Cruiser',   'Orion Arm',   205, 2226, 22.15);
```

1. SELECT - reading the registry · SELECT - читання реєстру

The officer asks for the full registry, then for a short manifest. The asterisk returns every column in the order declared in the DDL; an explicit column list returns only the requested columns and controls their order. AS assigns an alias that changes the heading of the output column but not the stored name.

Диспетчер запитує повний реєстр, а потім скорочений маніфест. Зірочка повертає всі стовпці у порядку, оголошеному в DDL; явний перелік стовпців повертає лише потрібні та задає їхній порядок. AS призначає псевдонім, який змінює заголовок вихідного стовпця, але не збережену назву.

```
-- 1.1 The complete registry / Повний реєстр
SELECT * FROM starship;

-- 1.2 A short manifest with aliases / Скорочений маніфест із псевдонімами
SELECT ship_name AS ship,
       ship_class AS class,
       crew_size AS crew
FROM starship;
```

2. SELECT DISTINCT - the reference lists · SELECT DISTINCT - довідкові переліки

Duplicate values appear in ship_class and home_sector because many ships share them. DISTINCT removes repeated rows from the result of the projection. When two columns follow DISTINCT, uniqueness applies to the pair, not to each column separately - query 2.2 therefore returns more rows than query 2.1.

Повторювані значення трапляються у стовпцях ship_class і home_sector, оскільки їх поділяють кілька суден. DISTINCT вилучає повторювані рядки з результату вибирання. Коли після DISTINCT указано два стовпці, унікальність застосовується до пари, а не до кожного стовпця окремо, тому запит 2.2 повертає більше рядків, ніж запит 2.1.

```
-- 2.1 Which ship classes are in service? / Які класи суден в експлуатації?
SELECT DISTINCT ship_class FROM starship;

-- 2.2 Which classes are based in which sectors? / Які класи базуються в яких секторах?
SELECT DISTINCT ship_class, home_sector
FROM starship;
```

3. ORDER BY - sorting the fleet · ORDER BY - впорядкування флоту

Without ORDER BY the server may return rows in any order, so the sequence of the output must never be assumed. ASC sorts ascending and is the default; DESC sorts descending. Several sort keys are applied from left to right: the second key orders only those rows that are equal on the first one.

Без ORDER BY сервер може повертати рядки у довільному порядку, тому послідовність виведення не можна вважати гарантованою. ASC упорядковує за зростанням і діє за замовчуванням; DESC - за спаданням. Кілька ключів сортування застосовуються зліва направо: другий ключ упорядковує лише ті рядки, що збігаються за першим.

```
-- 3.1 The largest crews first / Спочатку найбільші екіпажі
SELECT ship_name, crew_size
FROM starship
ORDER BY crew_size DESC;

-- 3.2 By class, and within a class from the newest ship / За класом, а в межах
-- класу - від найновішого судна
SELECT ship_name, ship_class, launch_year
FROM starship
ORDER BY ship_class ASC, launch_year DESC;
```

4. The LIMIT clause in MySQL · Конструкція LIMIT у MySQL

LIMIT alone takes the first n rows; the two-argument form LIMIT n OFFSET m skips m rows first, which is the basis of page-by-page output.

Сам по собі LIMIT повертає перші n рядків; форма з двома аргументами LIMIT n OFFSET m спершу пропускає m рядків, що є основою посторінкового виведення.

```
-- 4.1 Three best-fuelled ships / Три судна з найбільшим запасом палива
SELECT ship_name, fuel_reserve
FROM starship
ORDER BY fuel_reserve DESC
LIMIT 3;

-- 4.2 The next three ships (rows 4 to 6) / Наступні три судна (рядки 4-6)
SELECT ship_name, fuel_reserve
FROM starship
ORDER BY fuel_reserve DESC
LIMIT 3 OFFSET 3;
```

Control questions · Контрольні питання

1. Why does DISTINCT over two columns usually return more rows than DISTINCT over one of them?
2. Which clause of a query guarantees the order of rows, and what happens when it is omitted?
3. What is the difference between a column alias and the column name stored in the table definition?

1. Чому DISTINCT за двома стовпцями зазвичай повертає більше рядків, ніж DISTINCT за одним із них?
2. Яка конструкція запиту гарантує порядок рядків і що відбувається, коли її пропущено?
3. Чим псевдонім стовпця відрізняється від назви стовпця, збереженої у визначенні таблиці?